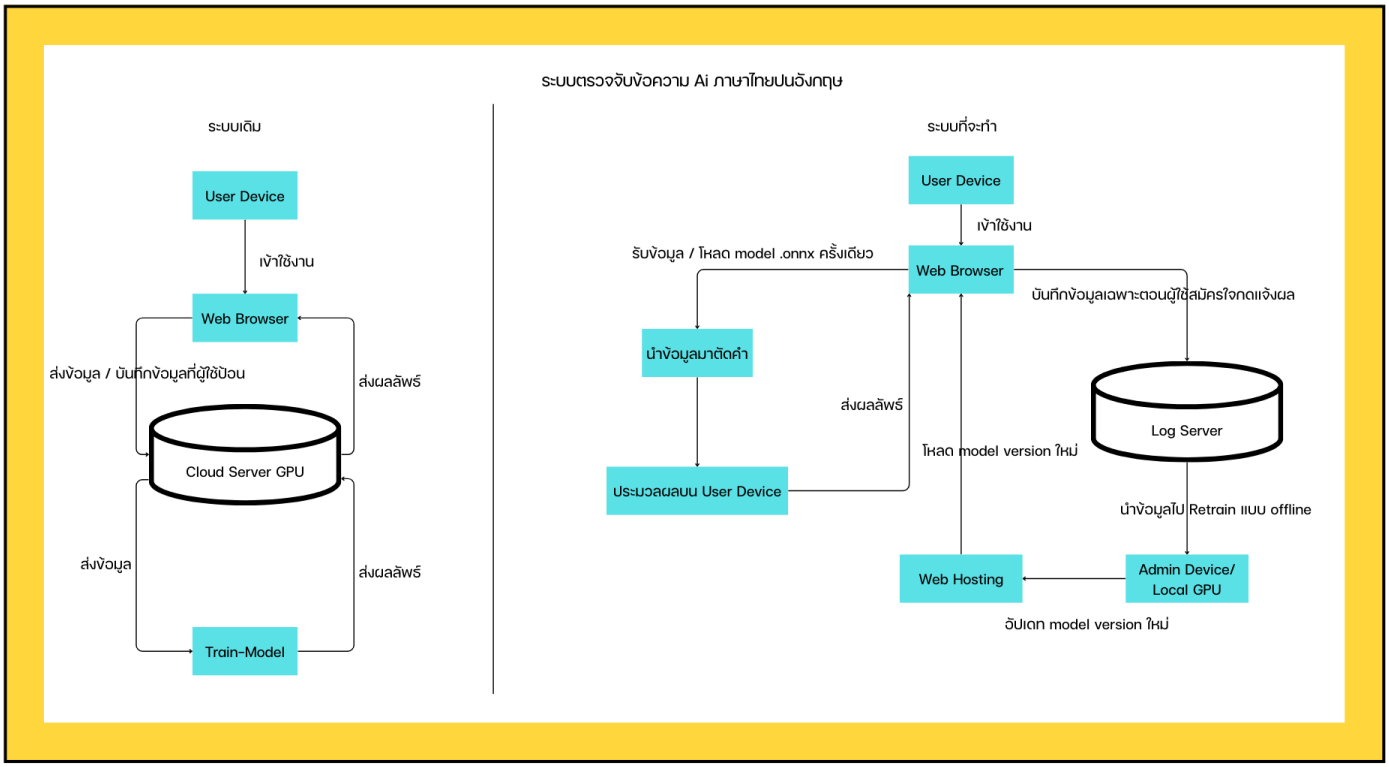
ระบบตรวจจับและระบุค่าย AI สำหรับข้อความภาษาไทยปนอังกฤษบนเว็บเบราว์เซอร์

ผู้จัดทำ: นายชิติพัทธ์ จำปา รหัสนักศึกษา 6609611857

สาขาวิชา: วิทยาการคอมพิวเตอร์

แผนการดำเนินงาน

1. **<u>แผนภาพเปรียบเทียบสถาปัตยกรรมระบบ</u>**



2. **<u>ตารางความเป็นไปได้และความท้าทาย</u>**

วัตถุประสงค์ของการเปรียบเทียบ: ตารางด้านล่างแสดงถึงการประยุกต์ใช้งานวิจัยและเทคโนโลยีต้นแบบผนวกเข้ากับ ความท้าทายทางวิศวกรรม (Engineering Challenges) ที่ผู้จัดทำได้ออกแบบเพิ่มเข้าไป เพื่อพิสูจน์ให้เห็นว่าโครงงานนี้มีความเป็นไปได้สูงในการพัฒนาให้สำเร็จ และมีจุดเด่นที่เหนือกว่าระบบดั้งเดิมอย่างชัดเจน โดยเฉพาะในด้านการรักษาความเป็นส่วนตัวของผู้ใช้ (Data Privacy) และการบริหารจัดการทรัพยากร (Resource Optimization)

องค์ประกอบหลัก	แหล่งอ้างอิง	ความท้าทายที่เพิ่มเข้าไป (Challenge)	ทำไมถึงมีโอกาสสำเร็จสูง (Feasibility)
1. โมเดลภาษาและการตรวจจับ	Wang et al. (2024) พิสูจน์ว่าโครงสร้าง BERT ตรวจจับข้อความ AI ได้แม่นยำถึง 99%	Language & Stylometry: งานวิจัยเดิมตัดคำเชื่อม (Stop words) ทิ้ง แต่โปรเจกต์นี้จะ รักษาคำเชื่อมภาษาไทยไว้ เพื่อเรียนรู้ความเยิ่นเย้อของ AI และต้องรองรับบริบทไทยปนอังกฤษ (Code-Switching)	ใช้ WangchanBERTa ซึ่งเป็นโมเดลที่ถูกพรีเทรนภาษาไทยมาอย่างดีแล้ว การนำมา Fine-tune ด้วยชุดข้อมูลที่เตรียมเอง
2. สถาปัตยกรรมการทำงาน	Ghostbuster (2023) รันโมเดลบน Cloud Server (Server-side) ขนาดใหญ่	Client-side Engineering: ย้ายการประมวลผลทั้งหมดมาไว้บน Web Browser โดยต้องบีบอัดโมเดล (Quantization) ให้มีขนาดเล็ก และเขียน JavaScript Tokenizer เพื่อตัดคำไทยบนหน้าเว็บ	ปัจจุบันมี ONNX Runtime Web ที่ทรงประสิทธิภาพ การแปลงโมเดล Hugging Face ไปเป็น ONNX มีไลบรารีรองรับชัดเจน
3. ความปลอดภัยของข้อมูล	ระบบ AI Detection ส่งข้อความผู้ใช้ไปประมวลผลและแอบบันทึกที่เซิร์ฟเวอร์	Zero-Data Leakage: สร้างระบบ Privacy-First ที่ข้อความจะไม่มีวันหลุดออกจากเครื่องผู้ใช้ ยกเว้นผู้ใช้จะกดปุ่ม "แจ้งผล" ด้วยตัวเอง	กระบวนการประมวลผลถูกล็อคให้อยู่ในฝั่ง Client 100% ทำให้ไม่ต้องยุ่งยากกับการเขียน API รับส่งข้อความที่ซับซ้อน
4. การเรียนรู้ของระบบ	Static Model เทรนเสร็จแล้วจบเลย โมเดลไม่อัปเดต	Offline Retraining Loop: การนำ Feedback ที่ผู้ใช้สมัครใจส่งมา กลับไป Retrain เพื่ออัปเดตโมเดลเวอร์ชันใหม่	แยกส่วนการเทรน (งานหนัก) ออกจากหน้าเว็บ ทำให้ระบบมีเสถียรภาพ ไม่ล่มง่าย

3. **<u>สถานที่ทำงานและการไหลของข้อมูล</u>**

1. โซนผู้พัฒนา (Admin Device / Local GPU)

- **ทำงานที่ไหน:** เครื่องคอมพิวเตอร์ของฝั่ง admin เอง
- **ทำงานยังไง (Offline):**
  - ใช้ประมวลผลงานหนัก คือการนำ Dataset (และข้อมูลจาก Log Server) มา Train / Retrain โมเดล
  - ทำการบีบอัดไฟล์โมเดลให้เป็นนามสกุล .onnx เพื่อให้มีขนาดเล็กลง โหลดเร็วขึ้น
- **การส่งข้อมูล:** อัปโหลดไฟล์โมเดลเวอร์ชันใหม่ (model.onnx) ขึ้นไปฝากไว้ที่ **Web Hosting**

2. โซนหน้าเว็บแอปพลิเคชัน (Web Hosting)

- **ทำงานที่ไหน:** เซิร์ฟเวอร์สำหรับฝากไฟล์เว็บไซต์และไฟล์โมเดล
- **ทำงานยังไง:** เป็นเพียงที่เก็บไฟล์ HTML/CSS/JS และไฟล์ .onnx รอให้ผู้ใช้เข้ามาเรียกใช้งาน
- **การส่งข้อมูล:** เมื่อผู้ใช้เปิดเว็บ Browser จะทำการ **ดาวน์โหลดโมเดลมาเก็บไว้ (Cache)** ในเครื่องผู้ใช้ครั้งเดียว

3. โซนผู้ใช้งาน (User Device / Web Browser)

- **ทำงานที่ไหน:** บนเบราว์เซอร์ในเครื่องคอมพิวเตอร์ หรือโทรศัพท์มือถือของผู้ใช้
- **ทำงานยังไง (Client-side):**
  - รับข้อความจากผู้ใช้ -> รัน JS Tokenizer ตัดคำ -> ส่งเข้าโมเดล ONNX ที่โหลดมาแล้ว -> แสดงผลลัพธ์ % ความเป็น AI
- **การส่งข้อมูล (จุดสำคัญ):**
  - **เวลาตรวจสอบปกติ:** ข้อมูลข้อความจะวิ่งวนอยู่แค่ในคอมพิวเตอร์ของผู้ใช้ **ไม่มีการส่งออกไปไหนทั้งสิ้น**
  - **เวลากดแจ้งผล (Feedback) :** ส่งข้อมูลแบบ JSON (เช่น {text: "...", label: "Human"}) วิ่งตรงไปที่ **Log Server**

4. โซนเก็บข้อมูลผู้ใช้ (Log Server)

- **ทำงานที่ไหน:** Cloud Server ขนาดเล็ก
- **ทำงานยังไง:** รอรับ Request ข้อมูลจากหน้าเว็บมาบันทึกลง Database
- **การส่งข้อมูล:** เป็นจุดพักข้อมูล รอให้ Admin ดึงข้อมูลนี้กลับไปเทรนที่เครื่อง Local GPU ในโซนที่ 1

<u>แหล่งอ้างอิง</u>

1. งานวิจัยด้านโมเดลภาษาไทย (Thai Language Model)

อ้างอิงเพื่อยืนยันเหตุผลที่เลือกใช้ WangchanBERTa เป็นโมเดลตั้งต้น เพราะเป็นโมเดลที่เข้าใจโครงสร้างภาษาไทยได้ดีที่สุด

- **ชื่อเปเปอร์:** WangchanBERTa: Pretraining transformer-based Thai Language Models (2021)
- **ผู้แต่ง:** Lalita Lowphansirikul, Charin Polpanumas, et al.
- **ลิงก์ (ArXiv):** https://arxiv.org/abs/2101.09635
- **ลิงก์ (PDF):** https://arxiv.org/pdf/2101.09635

2. งานวิจัยด้านอัลกอริทึมการตรวจจับ AI (AI Detection & Attribution)

อ้างอิงเพื่อยืนยันว่าการใช้สถาปัตยกรรม BERT และการวิเคราะห์ค่าความน่าจะเป็น (Probability) สามารถแยกแยะ AI ได้จริง

- **อ้างอิงหลัก (ปี 2024):** AI-Generated Text Detection and Classification Based on BERT Deep Learning Algorithm
  - **ผู้แต่ง:** Hao Wang, Jianwei Li, Zhengyu Li
  - **ลิงก์ (ArXiv):** https://arxiv.org/abs/2405.16422
  - **ลิงก์ (PDF):** https://arxiv.org/pdf/2405.16422
- **อ้างอิงเสริม (Ghostbuster):** Ghostbuster: Detecting Text Ghostwritten by Large Language Models (2023)
  - **ผู้แต่ง:** Vivek Verma, Eve Fleisig, Nicholas Carlini, et al.
  - **ลิงก์ (ArXiv):** https://arxiv.org/abs/2305.15047
- **อ้างอิงเสริม (การแยกค่าย AI - TuringBench):** TuringBench: A Benchmark Environment for Turing Test in the Age of Neural Text Generation (2021)
  - **ผู้แต่ง:** Adaku Uchendu, et al.
  - **ลิงก์ (ACL Anthology):** https://aclanthology.org/2021.emnlp-main.397/

3. งานวิจัยด้านสถาปัตยกรรม Client-side ML (Web Browser Processing)

อ้างอิงเพื่อสนับสนุนความท้าทายในการบีบอัดโมเดลให้มารันบนหน้าเว็บ เพื่อความรวดเร็วและรักษา Privacy ของผู้ใช้

- **ชื่อเปเปอร์:** TensorFlow.js: Machine Learning for the Web and Beyond (2019)
- **ผู้แต่ง:** Daniel Smilkov, Nikhil Thorat, et al.
- **ลิงก์ (ArXiv):** https://arxiv.org/abs/1901.05350
- **เครื่องมืออ้างอิง (ONNX Runtime Web):**
  - **ลิงก์ Document:** https://onnxruntime.ai/docs/tutorials/web/

4. แหล่งอ้างอิงโค้ดต้นแบบ (Implementation / Code Reference)

อ้างอิงเพื่อแสดงให้เห็นว่าได้ศึกษาแนวทางการเขียนโค้ดและทดลองจากโปรเจกต์จริง แล้วนำมาพัฒนาต่อยอด

- **ชื่อโปรเจกต์:** LLM - Detect AI Generated Text
- **ผู้พัฒนา:** Vidhi1290 (GitHub)
- **ลิงก์ Repository:** https://github.com/Vidhi1290/LLM---Detect-AI-Generated-Text

5. เครื่องมือเสริม: ระบบ OCR แบบ Client-side

อ้างอิงเผื่อในกรณีที่ใส่ฟีเจอร์ถ่ายรูปเพื่อแปลงเป็นข้อความบนหน้าเว็บ (โดยไม่ส่งรูปขึ้นเซิร์ฟเวอร์)

- **ชื่อโปรเจกต์:** Tesseract.js (Pure Javascript OCR for 100 Languages)
- **ผู้พัฒนา:** Naptha
- **ลิงก์ Repository:** https://github.com/naptha/tesseract.js